RETRIEVAL-AUGMENTED GENERATION (RAG) BASED PDF CHATBOT

Swaraj Shedge

INTRODUCTION

In contemporary enterprise and academic settings, the volume of unstructured data stored in Portable Document Format (PDF) files is growing rapidly. Extracting precise information from long, complex manuals, reports, and academic papers remains a time-consuming manual task. While Large Language Models (LLMs) demonstrate remarkable capabilities in natural language understanding, reasoning, and generation, their application to proprietary or custom documents is restricted by two main constraints: the context window limit of the model and the risk of hallucination—where the model generates factually incorrect but plausible-sounding answers due to the lack of access to ground-truth reference materials.

To address these limitations, this project implements a Retrieval-Augmented Generation (RAG) system. RAG is an architectural pattern that integrates an information retrieval component with a generative model. Instead of relying solely on the static knowledge encoded in the LLM's weights during training, the system dynamically retrieves semantically relevant text segments from a local document database and prepends them to the user’s prompt as context. This ensures that the generated responses are grounded in the uploaded documents.

Furthermore, this system is designed to operate entirely locally. By using local embeddings (BAAI/bge-base-en-v1.5) and local LLM execution via Ollama (llama3.1:8b), sensitive document data remains inside the host infrastructure, ensuring data privacy and security. The system also includes advanced retrieval techniques, such as multi-query expansion and a two-stage retrieval pipeline (reranking), optimized via Optuna and systematically evaluated using the Ragas framework.

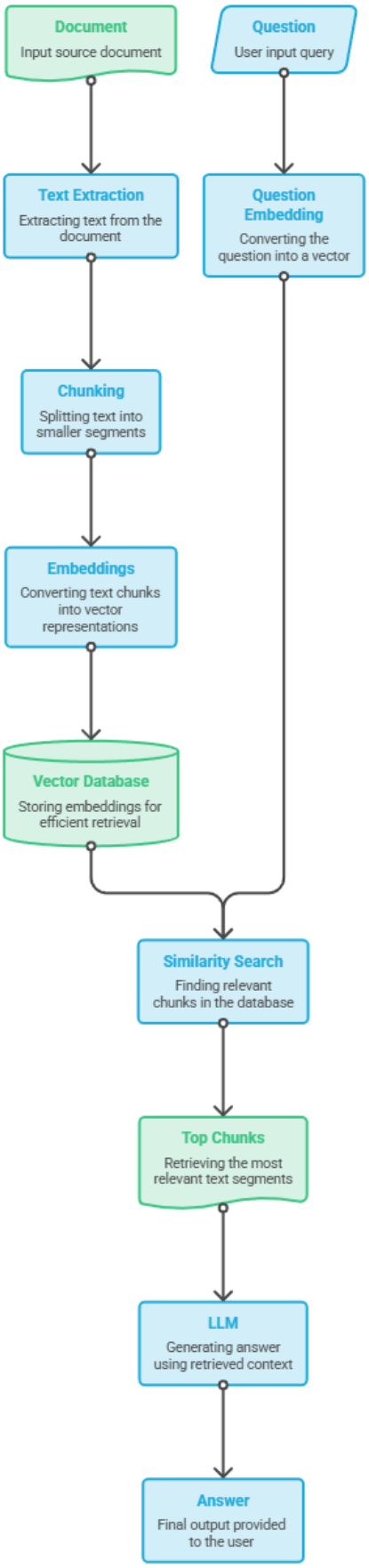
Problem Statement:

Standard deployments of Large Language Models (LLMs) present several critical operational challenges when applied to proprietary or localized documents:

1. **Hallucinations and Factuality:** Generative models lack a mechanism to verify facts from specific documents. When asked about domain-specific concepts, they tend to extrapolate, leading to incorrect or misleading answers.
2. **Lack of Domain Adaptation:** Fine-tuning an LLM on custom datasets is computationally expensive, requires large amounts of labeled data, and is difficult to update when documents change.
3. **Context Window Constraints:** Uploading complete textbooks or multi-page documents directly into an LLM's prompt window is either unsupported by local models or leads to performance degradation and high memory usage (known as the "lost in the middle" phenomenon).
4. **Data Privacy and Sovereignty:** Utilizing external commercial LLM APIs requires sending proprietary, personal, or corporate documents to external servers, which violates data privacy regulations (e.g., GDPR, HIPAA) and corporate policies.
5. **Lack of Traceability:** Standard LLM outputs do not provide citations, making it difficult for users to verify the statements back to their source document pages.

Therefore, there is a clear need for a software system that processes user-uploaded PDFs, stores their semantic representations securely in a local database, retrieves relevant information based on natural language queries, generates concise answers, and provides verifiable page citations—all while running within a secure, local, and resource-efficient environment.

General Flow for RAG Application:



OBJECTIVES:

The primary objectives of this project are:

1. **Develop a Local Document Ingestion Pipeline:** Implement a pipeline to extract text from multi-page PDF documents, perform header/footer and page-number cleaning, and split the text into semantic, overlapping chunks.
2. **Implement Semantic Vector Search:** Generate high-quality vector representations (embeddings) of document chunks and store them in a local relational database supporting vector operations.
3. **Design a Two-Stage Retrieval Architecture:** Improve retrieval precision by combining vector similarity search with query expansion (generating query variations using a local LLM) and a secondary re-ranking stage (using a Cross-Encoder model).
4. **Establish Secure Local Generation:** Deploy local inference for the LLM using Ollama to synthesize context-grounded answers.
5. **Expose Restful APIs and User Interface:** Build a modular backend using FastAPI and a reactive front-end interface using Streamlit to facilitate document upload, processing, and multi-turn chat interaction.
6. **Automate Evaluation and Hyperparameter Optimization:** Use the Ragas framework to measure system performance (Faithfulness, Relevance, Recall, Correctness) and apply Optuna to find the optimal set of RAG configurations (chunk size, overlap, temperature, top-k, and reranking parameters).

TECHNOLOGY STACK

The technology stack was selected to achieve a fully local, open-source, and high-performance system. The table below details each component, its purpose, and the rationale for its selection.

Technology	Purpose	Reason for Selection
Python 3.10+	Core Programming Language	Strong ecosystem for machine learning, natural language processing, and web development.
FastAPI	Backend Web API	Asynchronous framework, automated OpenAPI documentation generation, and high throughput compared to Flask.
Streamlit	Frontend UI	Rapid development of interactive UI components, native Python integration, and efficient handling of session states.
PostgreSQL	Relational Database Management System	Robust, ACID-compliant database capable of handling transactional data alongside vector search.
pgvector	Vector Similarity Search extension	Allows storing and querying high-dimensional vectors directly in PostgreSQL using SQL, eliminating the need for a separate standalone vector database.
Sentence Transformers (BAAI/bge-base-en-v1.5)	Embedding Generation	State-of-the-art open-source text embedding model. Balances vector size (768 dimensions) and retrieval accuracy on benchmarks.
Ollama	Local LLM Engine	Standardized local server for running LLMs, offering GGUF model management, GPU acceleration, and OpenAI-compatible endpoints.

Llama 3.1 (8B)	Generative LLM	Excellent reasoning capability, strong instruction-following for context containment, and runnable on consumer-grade hardware.
Optuna	Hyperparameter Tuning	Sequential model-based optimization (Bayesian optimization) framework for efficiently exploring hyperparameter spaces.
Ragas	Evaluation Framework	Evaluates RAG pipelines without requiring extensive manual test datasets by using LLM-as-a-judge metrics.
PyPDF2	PDF Text Extraction	Lightweight library for reading and extracting raw text from PDF files.

Pipelines in RAG Project

1.Document Ingestion Pipeline (Upload & Indexing)

This pipeline extracts, cleans, chunks, embeds, and stores PDF data in a PostgreSQL database using the pgvector extension.

Step 1: PDF Upload

- The user uploads one or more PDF files through the Home.py Streamlit interface.
- Streamlit sends a multipart/form-data POST request to the /upload endpoint defined in main.py.

Step 2: PDF Processing

- The backend stores the uploaded PDF inside the uploads/ directory.
- It then calls the extract_pages_from_pdf() function in pdf_processor.py to extract text page by page.

Step 3: Text Cleaning

Before chunking, the extracted text is cleaned by:

- Removing recurring headers and footers using the Counter class to detect repeated text across pages.
- Filtering standard page numbering patterns (e.g., *"Page 1 of 5"*) using regular expressions.
- Preserving the actual document content while removing unnecessary text.

Step 4: Text Chunking

- The cleaned pages are divided into smaller semantic chunks using LlamaIndex's SentenceSplitter.
- Default configuration:
    - Chunk Size: 500 characters
    - Chunk Overlap: 60 characters
- Overlapping chunks help preserve context across chunk boundaries, improving retrieval quality.

Step 5: Embedding Generation

- The generated chunks are passed to the store_chunks() function in rag.py.

- This function calls get_embeddings() from embeddings.py.
- A locally hosted Hugging Face SentenceTransformer model (BAAI/bge-base-en-v1.5) converts every chunk into a 768-dimensional embedding vector.

Step 6: Database Storage

The embeddings and metadata are stored in PostgreSQL using the pgvector extension.

Each record contains:

- Filename
- Page number
- Chunk index
- Chunk content
- 768-dimensional embedding vector (vector(768))

The documents are now indexed and ready for semantic retrieval.

2.Retrieval & Generation (RAG) Pipeline (Chatting)

This pipeline converts a user's question into matching document search results, refines them, and queries the LLM for a contextual response.

Step 1: User Question

- The user enters a question through the Chat.py Streamlit interface.
- Streamlit sends the query to the /chat endpoint in main.py.
- This endpoint invokes the answer_question() function in rag.py.

Step 2: Query Expansion

- Inside retrieve_relevant_chunks(), the system first calls expand_query_multi().
- A local Ollama LLM generates three alternative versions of the user's question.
- These expanded queries include synonyms and different phrasings, increasing retrieval coverage and improving the chances of finding relevant information.

Step 3: Vector Similarity Search

- The original query and all expanded queries are converted into embedding vectors using the same embedding model used during indexing.

- For each embedding, PostgreSQL with pgvector performs cosine similarity search using the <=> operator to retrieve the nearest document chunks.
- Similarity scores are computed as:

$$\text{Similarity} = 1.0 - \text{Cosine Distance}$$

- Duplicate chunks are removed based on their text content to avoid redundant context.

Step 4: Cross-Encoder Reranking

- The retrieved chunks are passed to the BAAI/bge-reranker-base Cross-Encoder model.
- Instead of comparing embeddings, the reranker evaluates each pair:

(User Question, Retrieved Chunk)

- It assigns a relevance score to every chunk.
- Chunks scoring below the configured RERANK_THRESHOLD (default: 0.35) are discarded.
- The highest-ranked chunks (default: Top 4) are selected as the final context.

Step 5: Prompt Construction

- The selected chunks are combined into a structured prompt.
- The prompt instructs the LLM to answer only using the provided context, reducing hallucinations and improving factual accuracy.

Step 6: Answer Generation

- The final prompt is sent to the local Ollama LLM.
- The model generates a response based solely on the retrieved document context.
- The answer, along with the corresponding source page numbers, is returned to the Streamlit frontend and displayed to the user.

DATABASE DESIGN

The database design uses PostgreSQL with the pgvector extension, allowing vector and relational data to be stored together in a single schema.

Table Schema: document_chunks

Below is the database schema description for the table that stores document text segments and their semantic embeddings.

Column Name	SQL Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing identifier for each chunk.
filename	TEXT	NOT NULL	The source filename of the uploaded PDF.
chunk_index	INTEGER	NOT NULL	The sequential index of the chunk within the document.
content	TEXT	NOT NULL	The actual text snippet extracted from the document.
embedding	vector(768)	None	The 768-dimensional embedding vector generated by the bge-base-en-v1.5 model.
page_number	INTEGER	None	The page number in the source PDF where the text snippet is located.

EVALUATION

Importance of Evaluation

Evaluating RAG pipelines is essential because standard metrics like BLEU or ROUGE only measure textual similarity, which does not reflect factual correctness, reasoning quality, or retrieval accuracy. Using an evaluation framework like Ragas provides a structured way to assess both retrieval quality and response generation accuracy.

Evaluation Dataset

The evaluation was performed using a validation dataset containing 40 test cases. Each test case consists of:

- A natural language question.
- A ground-truth answer (expected answer).
- Specific keywords to check for retrieval validity.

The evaluation process runs each question through the retrieval and generation pipeline, compiles the generated response and retrieved context blocks, and evaluates them across the Ragas metrics.

Ragas Metrics Explained

The system is evaluated using four core metrics:

1. **Faithfulness (0.0 to 1.0):** Measures the factual consistency of the generated answer against the retrieved context. It determines if the LLM hallucinated information not present in the reference documents.
2. **Answer Relevance (0.0 to 1.0):** Assesses how well the generated answer addresses the question. A low score indicates tangential or incomplete answers.
3. **Context Recall (0.0 to 1.0):** Evaluates the quality of the retrieval component by checking if all ground-truth facts are present in the retrieved chunks.
4. **Answer Correctness (0.0 to 1.0):** Measures the semantic and factual similarity between the generated answer and the ground truth.

Evaluation Results

Below is the summary of the evaluation runs compiled over the 40 test cases:

Metric	Score	Key Takeaways & Interpretations
Faithfulness	0.6548	Indicates that the generated answers are generally factual and grounded in the retrieved text, but highlights occasional issues where the LLM introduced outside knowledge or made minor logical leaps.
Answer Relevance	0.7532	Shows that the model effectively addresses the user's questions, aided by prompt structures that restrict response lengths and focus on the question.
Context Recall	0.6905	Reflects retrieval performance, showing that the vector database returned the correct pages in approximately 69% of the test cases. Performance was sometimes affected by complex phrasing.
Answer Correctness	0.6265	Reflects the overall accuracy of the system by comparing the generated answers directly with the ground-truth data, showing solid performance for a locally hosted setup.

Evaluation Module Workflow:

Load questions → Set up models → For each question: Retrieve chunks → Generate answer → Score with Ragas → Save results → Average all scores → Save CSV + JSON → Print summary

Evaluation Comparison of Different Models

Embedding Model: nomic embed text

Llm models: llama3.2 3B

```
========================================
        EVALUATION SUMMARY
========================================
  Total Questions   : 36
  Faithfulness      : 0.1894
  Answer Relevance  : 0.4228
  Context Recall    : 0.5655
  Answer Correctness: 0.4766
----------------------------------------
  Avg Latency       : 12.19s
  Min Latency       : 2.102s
  Max Latency       : 18.629s
========================================
```

Embedding model: BAAI/bge-base-en-v1.5

Llm model: llama3.1 8b

```
====================
EVALUATION SUMMARY
====================
Total Questions   : 40
Faithfulness      : 0.6548
Answer Relevance  : 0.7532
Context Recall    : 0.6905
Answer Correctness: 0.6265
--------------------
Avg Latency       : 33.755s
Min Latency       : 22.757s
Max Latency       : 73.918s
====================
```

Reranking Architecture

1.Overview

In a standard RAG pipeline, the initial retrieval phase uses embedding-based dense retrieval (vector similarity search via cosine distance) to query candidate chunks from the database. While efficient, vector similarity focuses primarily on semantic overlap and can sometimes retrieve noise or fail to capture fine-grained query context.

To overcome this, this project implements a two-stage retrieval pipeline:

1. Stage 1 (Coarse Retrieval): Retrieve a larger pool of candidate chunks using dense embeddings (BAAI/bge-base-en-v1.5) and PostgreSQL vector distance matching.
2. Stage 2 (Fine Re-ranking): Score and re-order the retrieved candidates using a specialized cross-encoder model to select the absolute best contexts for the LLM prompt.

2.Reranker Model Details

- **Model Used**: BAAI/bge-reranker-base (via sentence-transformers.CrossEncoder)
- **Type**: Cross-Encoder (determines similarity by feeding the query and document *together* into the transformer model, allowing full attention interaction between query and document tokens).
- **Execution Flow**:
    1. If reranking is enabled (use_rerank = True), the database retrieval limit is expanded to a wider pool: db_limit = max(top_k * 3, 10) to ensure high recall.
    2. Question-chunk pairs are constructed: [[question, chunk_content], ...]
    3. The CrossEncoder model outputs a scalar relevance score for each pair.
    4. The candidates are sorted in descending order of the cross-encoder score.
    5. A relevance threshold filter (RERANK_THRESHOLD, default: 0.35) is applied. Any chunk scoring below this threshold is discarded to prevent context pollution.

6. The top rerank_top_n chunks are sliced and returned as the final context block.

Hyperparameter Tuning

1.Overview

RAG pipeline performance is highly sensitive to parameters such as text chunking sizes, prompt temperature, and retrieval constraints. This project implements a dynamic, automated optimization suite using Optuna to optimize these hyperparameters against ground truth validation data.

Optimization Target (Objective Function)

The optimization suite evaluates pipeline performance using Ragas (Retrieval Augmented Generation Assessment) metrics. The objective is to maximize a combined utility score calculated as the average of three primary metrics:

Combined Score = (Faithfulness + Answer Relevancy + Answer Correctness)/3

- **Faithfulness**: Evaluates whether the generated answer is strictly grounded in the retrieved context (combating hallucinations).
- **Answer Relevancy**: Evaluates how well the generated answer addresses the actual question (avoiding noncommittal or generic answers).
- **Answer Correctness**: Evaluates the semantic similarity and factual accuracy of the generated answer against the ground truth dataset.